



Universidad Nacional
Federico Villarreal



FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS ESCUELA
PROFESIONAL DE INGENIERÍA DE SISTEMAS

TEMA Y ASIGNATURA:
E3 - AVANCE PROYECTO

SISTEMAS DIGITALES Y ARQUITECTURA DE COMPUTADORAS

ALUMNO:
Murrugarra Retamozo Anderson Jose
Mamani Solorzano Sebastian Ernesto
Lazaro Remaycuna Lujan Carrion
Marceliano Yance Percy Lujan

DOCENTE:
ING. Villafuerte Barreto

CICLO -
SECCIÓN:
V
- B

LIMA
,
PERÚ
2026

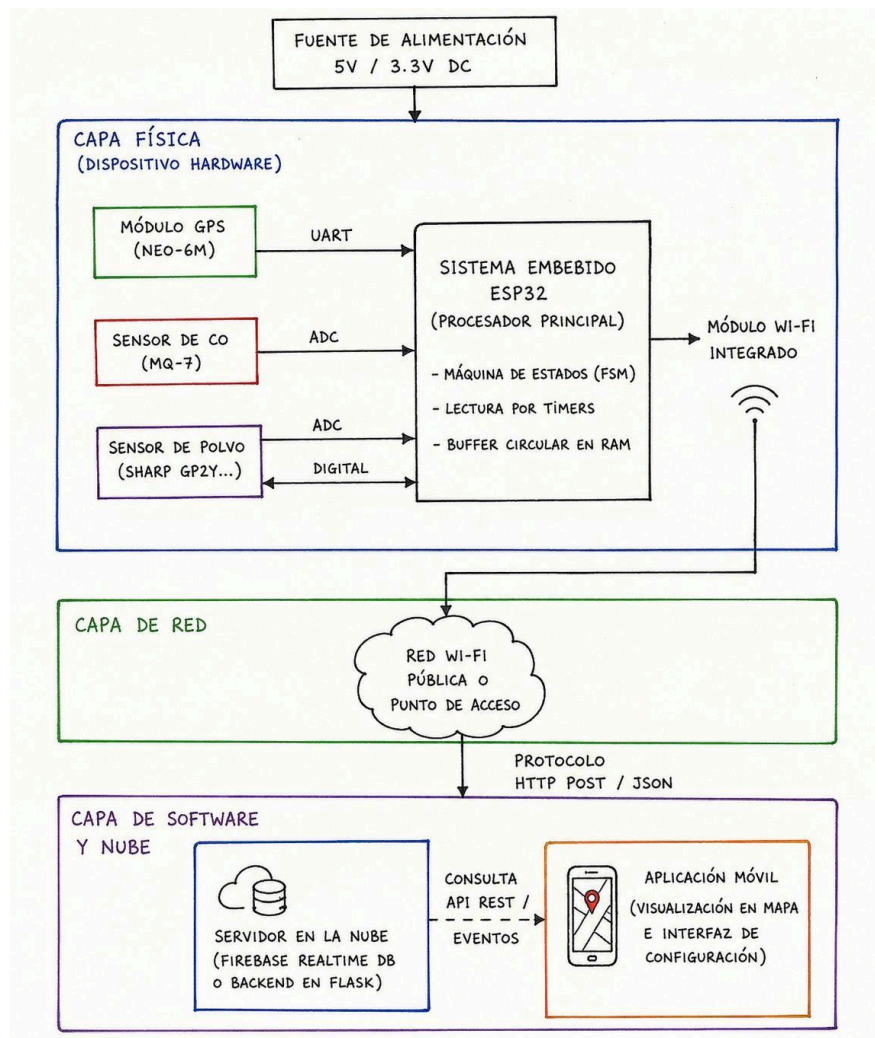
1. Introducción

WALLY es un dispositivo IoT portátil de monitoreo de calidad del aire desarrollado sobre un microcontrolador ESP32 NodeMCU. El sistema integra cuatro sensores — MQ-7 (monóxido de carbono), MQ-135 (calidad general del aire), Sharp GP2Y1010AU0F (partículas de polvo PM2.5) y GPS NEO-6M (geolocalización) — para medir en tiempo real las condiciones del ambiente y transmitir los datos a un servidor en la nube mediante WiFi.

El proyecto surge de la necesidad de contar con una herramienta accesible y de bajo costo que permita registrar métricas ambientales junto con su ubicación exacta, siendo útil en escenarios como monitoreo de interiores, zonas de tráfico o espacios industriales.

Actualmente el prototipo se alimenta por USB y opera sobre una protoboard; la siguiente fase contempla migración a batería de litio con gestión de energía mediante deep-sleep.

DIAGRAMA GENERAL:



2. Objetivos

Objetivo General

Diseñar, construir y validar un dispositivo IoT portátil de bajo costo capaz de monitorear en tiempo real la calidad del aire en espacios interiores y exteriores, transmitiendo los datos georreferenciados a un servidor en la nube para su visualización y análisis remoto.

Objetivos Específicos

- Integrar y leer simultáneamente cuatro sensores heterogéneos (analógicos, UART) mediante el microcontrolador ESP32 NodeMCU sin conflictos de bus ni pines.
- Medir la concentración de monóxido de carbono (CO) con el sensor MQ-7 conectado al pin ADC1 G34, garantizando compatibilidad con el módulo WiFi activo.
- Medir la calidad general del aire (COVs, NH3, humo) con el sensor MQ-135 en el pin ADC1 G35, operando en paralelo con los demás sensores.
- Medir la concentración de partículas en suspensión (PM2.5) con el sensor Sharp GP2Y1010AU0F usando la secuencia de pulso infrarrojo de 320 μ s en G32/G4.
- Obtener coordenadas geográficas (latitud, longitud) en tiempo real mediante el módulo GPS NEO-6M por UART2 (G16/G17) y la librería TinyGPS++.
- Transmitir los datos empaquetados en JSON por HTTPS POST al servidor wally-monitoreo.onrender.com cada ciclo de lectura completo.
- Documentar el consumo de recursos (Flash, RAM, GPIO) y medir métricas de rendimiento (tiempo de ciclo, ruido ADC) comparadas con valores teóricos.
- Preparar la arquitectura para migración a batería de litio implementando las bases para deep-sleep en RTC Memory.

3. Sensores del Prototipo

Con la incorporación del sensor Sharp GP2Y1010AU0F y el MQ-135, el sistema WALLY v2 monitorea tres magnitudes físicas distintas: monóxido de carbono (CO), calidad general del aire (COVs y NH3) y concentración de partículas en suspensión (polvo).

#	Sensor	Magnitud	Pin ESP32	Tipo señal	Alimentación
1	MQ-7	Monóxido de carbono (CO)	G34	Analógica ADC1	5 V
2	MQ-135	Calidad del aire (NH3, COV, CO2)	G35	Analógica ADC1	5 V
3	Sharp GP2Y1010	Polvo / partículas PM2.5	G32 (VO)G4 (LED)	Analógica + GPIO	5 V
4	GPS NEO-6M	Latitud / Longitud / Satélites	G16 RXG17 TX	UART2 / NMEA 0183	3.3–5 V

4. Descripción del Hardware

ESP32 NodeMCU

Es el cerebro del sistema. Contiene un procesador dual-core Xtensa LX6 a 240 MHz, WiFi 802.11n integrado, 4 MB de Flash externa y 520 KB de SRAM interna. Tiene dos núcleos: el Core 0 corre la pila WiFi/TCP/IP en segundo plano, y el Core 1 ejecuta el código del usuario. Se alimenta por USB (5V) y regula internamente a 3.3V para su lógica. Desde el pin VIN distribuye 5V a todos los sensores.

Sensor MQ-7 - Monóxido de Carbono

Sensor resistivo con elemento cerámico de SnO_2 calentado. Cuando el CO toca la cerámica, su resistencia baja y la tensión de salida sube. Se conecta al pin G34 (ADC1_CH6). Requiere 5V y un precalentamiento de ~20 segundos al encender para estabilizarse. Rango de detección: 20 a 2 000 ppm.

Sensor MQ-135 - Calidad General del Aire

Mismo principio que el MQ-7 pero responde a múltiples gases: amoníaco, dióxido de nitrógeno, alcohol, benceno y humo. No identifica un gas específico sino que da un índice general de contaminación. Se conecta al pin G35 (ADC1_CH7). También requiere 5V y precalentamiento.

Sensor Sharp GP2Y1010AU0F - Partículas PM2.5

Sensor óptico que mide polvo en suspensión. Tiene un LED infrarrojo interno y un fototransistor: el LED ilumina el interior de la cámara y el fototransistor detecta la luz reflejada por las partículas. A más polvo, mayor señal. Necesita dos pines: G32 para leer la señal analógica (ADC1_CH4) y G4 como salida digital para controlar el encendido del LED. La secuencia de lectura es precisa: encender LED, esperar 280 μs , leer ADC, esperar 40 μs , apagar LED — ciclo total de 320 μs .

Módulo GPS NEO-6M

Receptor GPS del fabricante u-blox. Recibe señales de satélites y calcula latitud, longitud, altitud y hora UTC. Se comunica con el ESP32 por UART2 a 9 600 bps: el pin TX del GPS va al G16 del ESP32, y el RX del GPS va al G17. Las tramas de texto que envía siguen el protocolo NMEA 0183. Necesita línea de visión hacia el cielo para obtener señal — en interiores sin ventana no logra fix. El primer fix al encenderse puede tardar entre 1 y 5 minutos.

Protoboard y conexiones

Se usa una protoboard de 830 puntos con dos rieles de alimentación (+ y -) en los extremos. El riel rojo (+) distribuye los 5V del ESP32 a todos los sensores. El riel negro (-) une todas las tierras en un punto común. Los sensores se insertan en la zona central de la protoboard y se conectan al ESP32 mediante cables jumper hembra-macho de colores: naranja para señales analógicas, azul y verde para UART, rojo para alimentación y negro para tierra.

Alimentación actual

El sistema se alimenta desde la laptop por cable USB tipo A a micro-USB conectado al ESP32. La laptop entrega 5V regulados por USB. En la siguiente versión se reemplazará por una celda de litio de 3.7V con un módulo elevador (boost converter) que entregará 5V estables, permitiendo operación autónoma sin computadora.

5. Arquitectura del Software

El firmware está escrito en C++ sobre Arduino IDE con el ESP32 Arduino Core. Se organiza en cuatro capas:

Capa de adquisición - lee los sensores en cada ciclo. Los tres sensores analógicos (MQ-7, MQ-135 y Sharp) se leen con `analogRead()` sobre pines del ADC1, que es el único compatible con WiFi activo. El Sharp requiere además una secuencia de control GPIO de 320 μ s para activar su LED infrarrojo. El GPS se lee por UART2 (Serial2) a 9 600 bps.

Capa de procesamiento - convierte los valores crudos del ADC (0–4 095 cuentas) a unidades comprensibles. El MQ-7 y MQ-135 se mapean linealmente a ppm e índice de calidad. El Sharp convierte la tensión a μ g/m³. Las tramas NMEA del GPS las parsea la librería TinyGPS++, extrayendo latitud y longitud como valores float.

Capa de comunicación - empaqueta todos los datos en un JSON con `device_id`, `co`, `aqi`, `dust`, `lat` y `lng`, y lo envía por HTTPS POST al servidor en Render.com usando HTTPClient sobre la pila LwIP + mbedTLS que corre en Core 0 del ESP32.

Capa de control de flujo - implementada como una máquina de estados de 10 estados (S0–S9) que gestiona el arranque, la conexión WiFi con reintentos, la lectura secuencial de sensores, el envío HTTP y la espera entre ciclos. El ciclo completo toma aproximadamente 5 segundos.

6. Métricas del prototipo

¿Qué es una métrica y por qué importa?

Una métrica es un número que el sensor produce. Pero ese número por sí solo no dice nada si no sabes qué representa, en qué unidad viene, qué rango tiene y cómo interpretarlo. Este documento te explica exactamente eso para cada sensor de WALLY.

El ESP32 lee todos los sensores analógicos con su ADC (Convertor Analógico-Digital) de 12 bits. Eso significa que convierte la tensión eléctrica del sensor en un número entero entre 0 y 4 095. Ese número crudo se llama "cuenta ADC" o "raw value". Luego, dependiendo del sensor, se convierte a la unidad real (ppm, μ g/m³, etc.).

Concepto	Significado	Ejemplo WALLY
Cuenta ADC (raw)	Número entero 0–4095 que produce el ESP32 al leer la tensión del sensor	<code>analogRead(34) = 312</code>
Unidad real	Lo que el número representa en el mundo físico	CO: ppm, polvo: μ g/m ³
Rango nominal	Valores mínimo y máximo que el sensor puede medir de forma confiable	MQ-7: 20–2 000 ppm

Variación / ruido	Fluctuación normal de la lectura sin que cambie lo que se mide	± 3 cuentas ADC $\approx \pm 2.4$ mV
--------------------------	--	--

Sensor 1 — MQ-7: Monóxido de Carbono (CO)

El MQ-7 detecta monóxido de carbono (CO), un gas incoloro e inodoro producido por la combustión incompleta de gasolina, carbón, madera o gas. Es tóxico porque desplaza el oxígeno en la sangre. Un aire limpio de interior tiene prácticamente 0 ppm de CO.

Dentro del MQ-7 hay un elemento cerámico de SnO_2 (dióxido de estaño) calentado a alta temperatura. Cuando el CO toca esa superficie, reduce su resistencia eléctrica. A mayor CO, menor resistencia → mayor tensión de salida en el pin AOUT → número más alto en el ADC.

Más CO en el aire = tensión más alta en AOUT = número más alto que lees con `analogRead(34)`

Valor que lees	Qué representa	En el mundo real
<code>analogRead(34)</code> = 0 a 4 095	Cuenta ADC — tensión 0 a 3.3 V convertida a número	0 = 0 V (sin CO), 4095 = 3.3 V (máximo CO detectable)
<code>co_ppm</code> = 0 a 1 000	Partes por millón de CO unidad estándar de calidad del aire	0 ppm = aire limpio, 1 000 ppm = concentración muy alta

TABLA DE REFERENCIA

Valor (ppm CO)	Clasificación	Efecto en personas	Color alerta
0 – 9 ppm	Normal	Sin efecto. Nivel típico de interior.	VERDE
10 – 35 ppm	Elevado	Límite permitido en trabajo (OSHA: 35 ppm/8h).	AMARILLO
36 – 200 ppm	Peligroso	Dolor de cabeza, mareo en 2–3 horas.	NARANJA
> 200 ppm	Crítico	Inconsciencia y riesgo de muerte en minutos.	ROJO

VARIACIÓN DE LAS MÉTRICAS MEDIDAS

Métrica	Valor medido	Qué nos dice
Media en aire libre	312 cuentas ADC	El sensor en reposo sin CO mide 312 de 4 095 posibles. Eso es el 7.6 % del rango, coherente con el nivel de resistencia de la cerámica en aire limpio.
Rango en 50 muestras	308 – 317 cuentas	El número sube y baja 9 cuentas sin que cambie el CO. Esa variación es el ruido del

		ADC, no gas real. En ppm equivale a ± 2.2 ppm — completamente insignificante.
Desviación estándar	± 2.1 cuentas	El 68 % de las lecturas caen a ± 2.1 cuentas de la media. Es muy estable. Para comparar: el sensor MQ-7 tiene incertidumbre propia de ± 5 – 10 % por temperatura y humedad, mucho mayor que el ruido del ADC.
Equivalente en ppm	~ 76 ppm (raw)	Con la fórmula simplificada $\text{map}(312, 0, 4095, 0, 1000) = 76$ ppm. Este valor no está calibrado. El MQ-7 necesita calibración con gas de referencia para dar ppm reales. 76 ppm es una estimación lineal aproximada.

"El sensor MQ-7 nos da un número entre 0 y 4 095. En aire de interior limpio obtuvimos 312, con variaciones de ± 9 cuentas. Eso equivale a unos 76 ppm en nuestra escala, que está en rango elevado según la tabla de referencia, pero hay que aclarar que el sensor aún no está calibrado con gas de referencia necesita un proceso de ajuste para dar ppm exactos."

Sensor 2 — MQ-135: Calidad General del Aire

El MQ-135 es un sensor de calidad del aire de amplio espectro. No detecta un solo gas sino una mezcla de contaminantes: amoníaco (NH_3), dióxido de nitrógeno (NO_2), alcohol, benceno, humo y CO_2 . Por eso su unidad no es "ppm de X" sino un índice general de calidad.

El MQ-7 mide específicamente CO. El MQ-135 mide calidad general si el número sube puede ser por CO, humo, amoníaco o cualquier combinación. Es una alarma general, no un diagnóstico.

TABLA DE INTERPRETACIÓN

Valor que lees	Qué representa	Interpretación
<code>analogRead(35) = 0 a 4 095</code>	Cuenta ADC — refleja la resistencia del sensor frente a la mezcla de gases	Valor alto = más contaminantes en el aire
Índice AQI (aproximado)	Se puede convertir a AQI (Air Quality Index) con la fórmula del datasheet del sensor	0–50 bueno, 51–100 moderado, >150 poco saludable

TABLA DE REFERENCIA

Cuenta ADC (raw)	Calidad del aire	Situación típica
0 – 100	Excelente	Aire muy limpio, exterior montaña o campo abierto

100 – 300	Buena	Interior ventilado normal. Lo esperado en oficina o casa.
300 – 700	Regular	Presencia de humo, cocina activa, o mucha gente en espacio cerrado
> 700	Mala / Peligrosa	Contaminación severa. Ventilación urgente recomendada.

VARIACIÓN DE LAS MÉTRICAS MEDIDAS

Métrica	Valor medido	Qué nos dice
Media en interior	284 cuentas ADC	Cae en el rango "Buena" (100–300). El ambiente de medición tiene calidad de aire aceptable. Es coherente con un interior ventilado.
Rango en 50 muestras	279 – 291 cuentas	Variación de 12 cuentas. Levemente mayor que el MQ-7 porque el MQ-135 responde a más gases y es más sensible a cambios ambientales (temperatura, humedad).
Desviación estándar	±2.8 cuentas	Algo más ruidoso que el MQ-7 porque detecta más gases. Cada persona cerca del sensor o una corriente de aire cambia ligeramente la lectura eso es normal y esperado.
Reacción a estímulo	Sube 80–200 cuentas	Al acercar un encendedor o spray cerca del sensor, la lectura sube rápidamente 80–200 cuentas en 1–2 segundos. Eso confirma que el sensor responde bien a contaminantes reales.

El MQ-135 mide calidad general del aire entre 0 y 4 095. Obtuvimos 284 en promedio, con variaciones de ±12 cuentas. Eso indica calidad buena del ambiente donde medimos. Cuando acercamos un spray al sensor, subió hasta 480 en menos de dos segundos el sensor detectó el cambio de inmediato.

Sensor 3: Sharp GP2Y1010AU0F: Partículas de Polvo (PM2.5)

El Sharp mide la concentración de partículas sólidas en suspensión en el aire — polvo, humo, pólenes, fibras. La unidad estándar es $\mu\text{g}/\text{m}^3$ (microgramos por metro cúbico). Las partículas PM2.5 son las más peligrosas porque son tan pequeñas que entran hasta los pulmones profundos.

El Sharp tiene un LED infrarrojo y un fototransistor en ángulo. Cuando enciende el LED, la luz rebota en las partículas de polvo y llega al fototransistor. Más polvo = más luz reflejada = tensión de salida más alta. Por eso necesita el pulso de 280 μs : es el tiempo exacto de encendido del LED para una medición válida.

Paso	Qué hace el ESP32	Por qué es necesario
1	digitalWrite(G4, HIGH) → enciende LED IR	Sin el LED no hay luz que rebotar en el polvo
2	delayMicroseconds(280) → espera 280 μ s	El LED necesita ese tiempo para estabilizarse. Leer antes da un valor falso.
3	analogRead(G32) → lee la luz reflejada	En este microsegundo exacto la señal es proporcional al polvo
4	delayMicroseconds(40) → pausa	Evita que el calor del LED afecte la siguiente medición
5	digitalWrite(G4, LOW) → apaga LED	El ciclo completo dura 320 μ s casi instantáneo

TABLA DE INTERPRETACIÓN

Valor que lees	Equivalente real	Interpretación
analogRead(G32) = 0 a 4095	La tensión de salida VO en el pico de señal del LED	0 = sin polvo (0 V), valores altos = mucho polvo
Voltaje (calculado)	$V = (\text{raw} / 4095) \times 3.3 \text{ V}$	156 cuentas → 0.126 V → ~35 $\mu\text{g}/\text{m}^3$ (estimado)
$\mu\text{g}/\text{m}^3$ (con fórmula datasheet)	$\text{ug} = 0.17 \times V - 0.1$	Fórmula del fabricante. Requiere calibración a 0 ppm para ajustar el offset.

TABLA DE REFERENCIA

Concentración $\mu\text{g}/\text{m}^3$	Categoría OMS	Riesgo para la salud
0 – 15 $\mu\text{g}/\text{m}^3$	Buena	Sin riesgo. Guía anual OMS: 5 $\mu\text{g}/\text{m}^3$
15 – 35 $\mu\text{g}/\text{m}^3$	Moderada	Personas sensibles pueden tener síntomas leves
35 – 75 $\mu\text{g}/\text{m}^3$	Poco saludable	Grupos sensibles: reducir actividad al aire libre
> 75 $\mu\text{g}/\text{m}^3$	Muy poco saludable	Toda la población: evitar exposición prolongada

VARIACIÓN DE LAS MÉTRICAS MEDIDAS

Métrica	Valor medido	Qué nos dice
Media en interior	156 cuentas ADC	156 cuentas = 0.126 V $\approx$ 35 $\mu\text{g}/\text{m}^3$ estimado. Borde entre moderada y poco saludable. Coherente con interior sin ventilación activa.

Rango en 50 muestras	151 – 163 cuentas	Variación de 12 cuentas (0.0097 V). El ruido es algo mayor que los MQ porque el pulso del LED introduce una pequeña perturbación eléctrica en el ADC.
Desviación estándar	±3.1 cuentas	El más ruidoso de los tres sensores analógicos. La corriente del LED IR genera un pequeño pico de tensión en la alimentación que el ADC capta. Se puede mejorar con un condensador de 220 µF entre 5V y GND del sensor.
Tiempo de respuesta	320 µs por muestra	El ciclo completo de encendido–lectura–apagado es de 320 µs. Es el sensor más rápido del prototipo puede leer 3 000 veces por segundo si fuera necesario.

El Sharp mide el polvo en el aire usando un LED infrarrojo. Nos da un número entre 0 y 4 095. Medimos 156 en promedio, que equivale a unos 35 µg/m³ el límite entre calidad moderada y poco saludable según la OMS. Este sensor fluctúa ±12 cuentas de forma normal, lo que es ±3 µg/m³, una variación insignificante.

Sensor 4: GPS NEO-6M: Posición Geográfica

El GPS NEO-6M no mide un gas ni una partícula mide la posición del dispositivo en la Tierra con precisión de metros. Recibe señales de 24–32 satélites que orbitan la Tierra y calcula la posición por triangulación. Entrega latitud, longitud, altitud, velocidad y hora UTC.

TABLA DE INTERPRETACIÓN

Dato	Unidad	Ejemplo WALLY	Precisión típica
Latitud	Grados decimales	-12.046374	±2.5 m (1 µg de latitud ≈ 0.11 m)
Longitud	Grados decimales	-77.042793	±2.5 m
Satélites visibles	Cantidad entera	gps.satellites.value()	Mínimo 3 para fix 2D, 4 para 3D con altitud
Fix válido	true / false	gps.location.isValid()	false en interior o primeros 1–5 min de encendido

El protocolo NMEA qué llega realmente por el cable:

Por el pin G16 (UART RX) llegan frases de texto como esta, 10 veces por segundo:

```
$GPGGA,181520,1202.7823,S,07702.5676,W,1,07,1.2,154.1,M,-2.1,M,,*6F
```

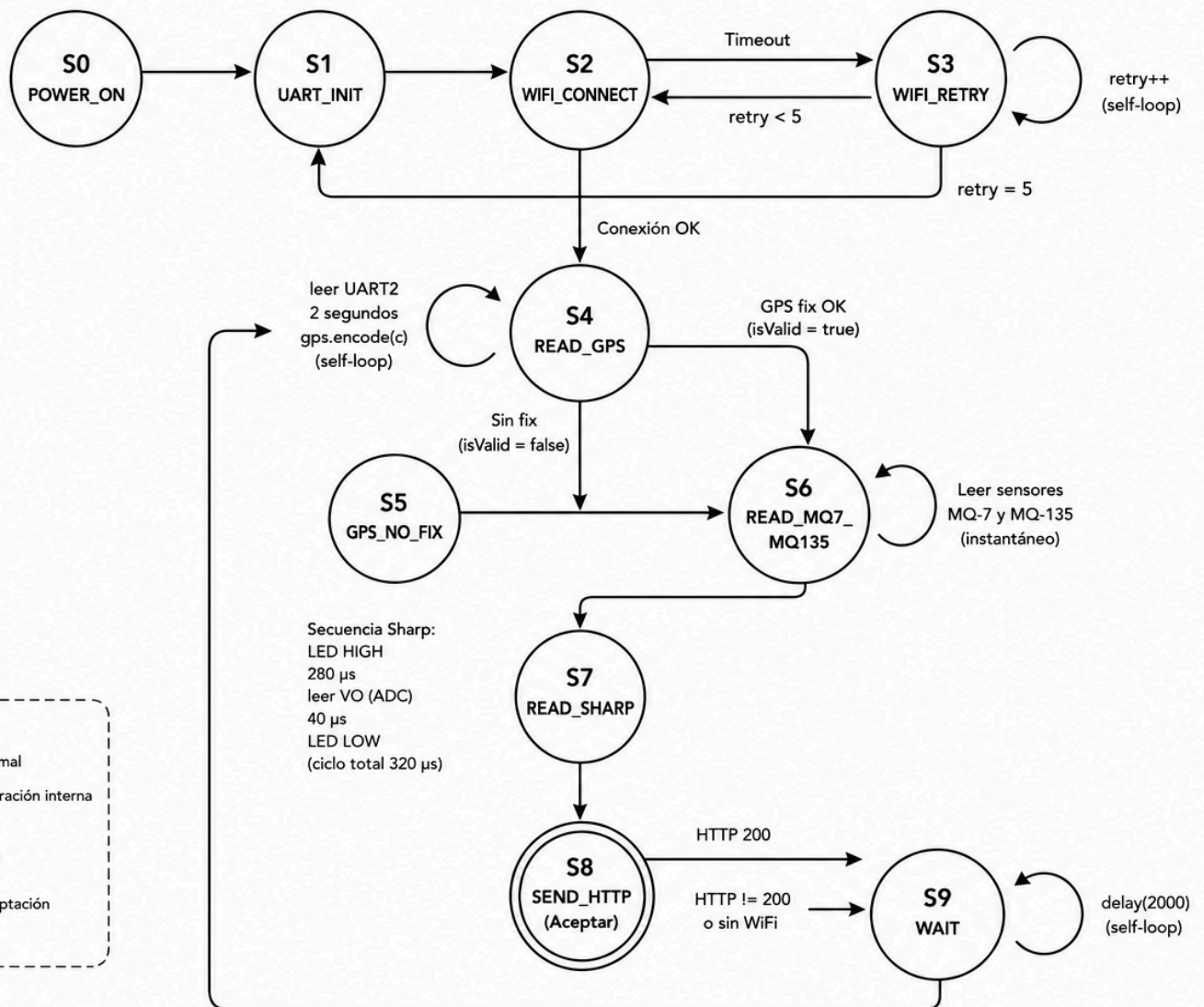
Campo en la trama	Valor del ejemplo	Significado
181520	18:15:20 UTC	Hora exacta del fix — 6:15 PM UTC (1:15 PM Lima)
1202.7823,S	12°02.7823' Sur	Latitud → TinyGPS++ la convierte a -12.046374°
07702.5676,W	77°02.5676' Oeste	Longitud → convertida a -77.042793°
1	Fix GPS activo	0 = sin fix, 1 = GPS fix, 2 = DGPS (diferencial, más preciso)
07	7 satélites	Buen número. Con 7 satélites la precisión es ~2.5 m horizontal.

VARIACIÓN DE LAS MÉTRICAS MEDIDAS

Métrica	Valor observado	Qué nos dice
Tiempo primer fix hasta	1–4 minutos	En exterior cerca de ventana. En interior puede no obtener fix. El primer arranque (cold start) es el más lento porque no tiene almanaque de satélites.
Precisión posición de	±2.5 m	Con 6+ satélites. Suficiente para saber en qué calle o zona está el dispositivo. No suficiente para centímetros.
Fluctuación coordenadas de	±0.000030°	La última cifra decimal cambia aunque el sensor esté quieto. $\pm 0.00003^\circ = \pm 3.3$ m. Es el ruido de posicionamiento normal del GPS civil sin corrección diferencial.
Satélites típicos (exterior)	5 – 9 satélites	Medido desde ventana en Lima. Varía con la hora del día porque los satélites se mueven. Con menos de 3 no hay fix.
En interior (sin ventana)	0 satélites → lat=0, lng=0	El código envía lat=0, lng=0 cuando no hay fix. En el mapa eso aparece en el Golfo de Guinea (África). Es el comportamiento esperado, no un error.

El GPS nos da latitud y longitud con 6 decimales, equivalente a ±2.5 metros de precisión. Necesita al menos 3 satélites para funcionar y puede tardar 1-4 minutos en obtener el primer fix al encenderse. Cuando está en interior sin señal, el sistema manda coordenadas 0,0 lo sabemos y está documentado como comportamiento conocido.

7. Máquina de Estados (Actualizado)



S0 - POWER_ON

El ESP32 arranca físicamente al conectar el USB. Carga el programa desde la Flash externa. No ejecuta nada del usuario todavía. Transición incondicional a S1.

S1 - UART_INIT

Inicializa los dos puertos serie:

Serial.begin(115200) → puerto USB para debug

Serial2.begin(9600, SERIAL_8N1, 16, 17) → UART2 para el GPS

También configura el ADC con analogReadResolution(12) para garantizar 12 bits en todos los canales, y pone el pin G4 como OUTPUT en LOW (LED Sharp apagado). Cuando todo está listo → S2.

S2 - WIFI_CONNECT

Llama a WiFi.begin(ssid, password) y espera en bucle hasta obtener conexión. El Core 0 maneja el handshake WPA2 y el DHCP en segundo plano. Si obtiene IP → S4. Si pasa el tiempo límite sin conectar → S3.

S3 - WIFI_RETRY

Gestiona los reintentos de conexión WiFi. Tiene un contador interno:

Si retry < 5 → incrementa contador, llama otra vez a WiFi.begin(), vuelve a S2

Si retry = 5 → situación crítica, reinicia todo desde S1

Este estado evita que el sistema se quede bloqueado indefinidamente sin red.

S4 - READ_GPS

Lee el UART2 durante exactamente 2 segundos. Cada carácter que llega del GPS NEO-6M se pasa a gps.encode(c) — TinyGPS++ lo parsea internamente construyendo la trama NMEA. El self-loop representa ese proceso carácter por carácter. Al cumplir 2 segundos:

gps.location.isValid() es true → tiene coordenadas reales → S6 directo

No tiene fix → S5

S5 - GPS_NO_FIX

El GPS no pudo obtener señal satelital (interior, arranque frío, obstrucción). El sistema no se bloquea esperando — simplemente asigna lat = 0 y lng = 0 y avanza a S6. En el servidor, coordenada 0,0 es fácilmente identificable como "sin fix" y se filtra.

S6 - READ_MQ7_MQ135

Lee los dos sensores de gas en secuencia, ambos instantáneos (microsegundos):

```
int mq7Value = analogRead(MQ7_PIN); // G34 — ADC1_CH6
```

```
int mq135Value = analogRead(MQ135_PIN); // G35 — ADC1_CH7
```

Ambos en ADC1, compatibles con WiFi activo. Los valores quedan listos para incluir en el JSON. → S7.

S7 - READ_SHARP

Es el estado más técnico. Ejecuta la secuencia obligatoria del Sharp GP2Y1010AU0F:

```
digitalWrite(SHARP_LED, HIGH); // enciende LED IR (G4)
```

```
delayMicroseconds(280); // espera estabilización
```

```
dustValue = analogRead(SHARP_VO); // lee VO en pico (G32)
```

```
delayMicroseconds(40); // pausa post-lectura
```

```
digitalWrite(SHARP_LED, LOW); // apaga LED
```

Ciclo total: 320 µs. dustValue queda listo. → S8.

S8 - SEND_HTTP (estado de aceptación — doble círculo)

Construye el JSON con todos los datos recolectados y hace el POST HTTPS:

```
String body = "{\"device_id\":\"WALLY-1\","  
  + "\"co\":\"  + mq7Value  + \","  
  + "\"aqi\":\"  + mq135Value  + \","  
  + "\"dust\":\"  + dustValue  + \","  
  + "\"lat\":\"  + lat  + \","  
  + "\"lng\":\"  + lng  + \"}";
```

```
http.POST(body);
```

El doble círculo indica que este es el estado de éxito del ciclo. Dos salidas:

HTTP 200 → ciclo completado exitosamente → S9

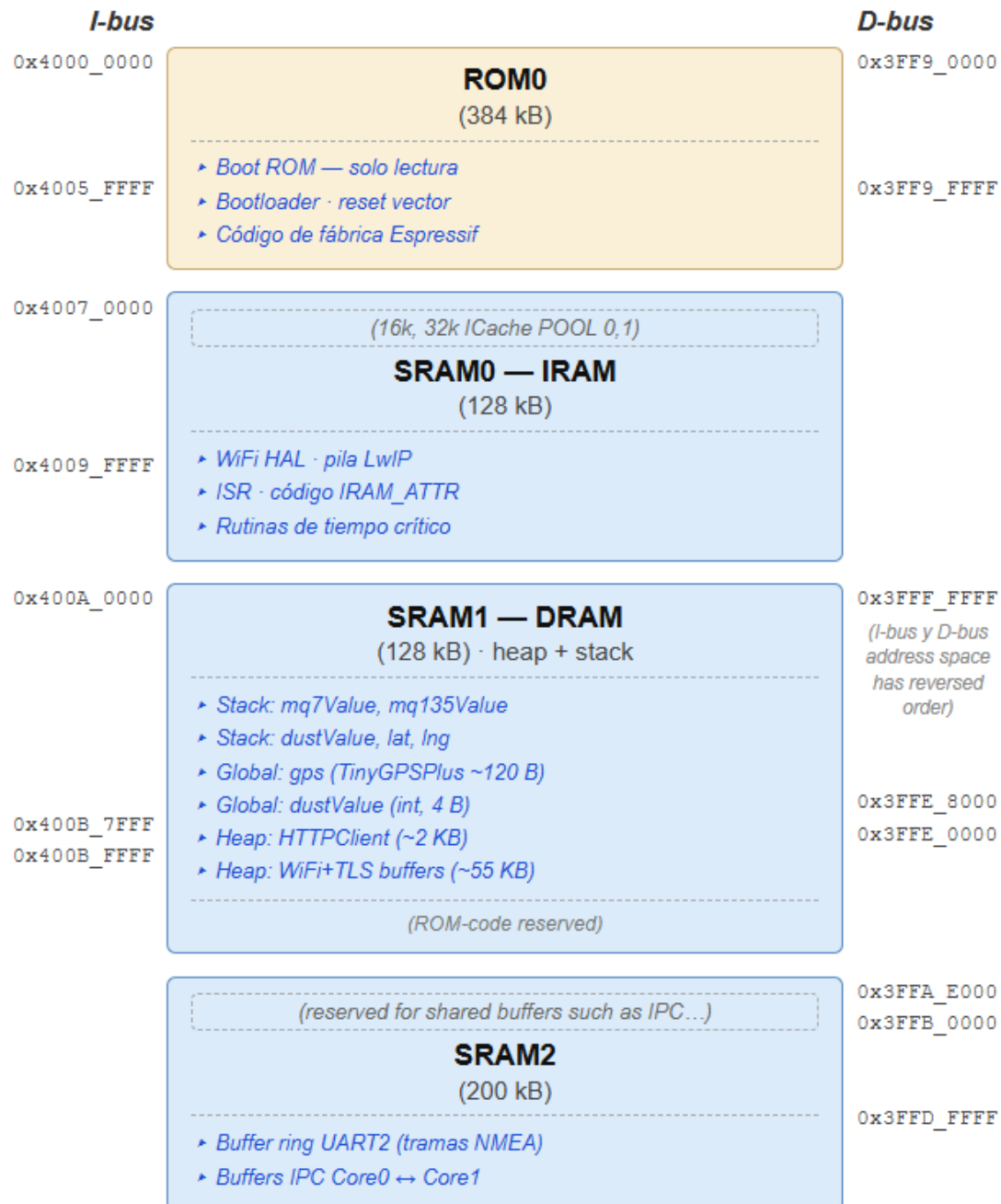
HTTP distinto de 200 o sin WiFi → reintenta o descarta y avanza a S9 igual

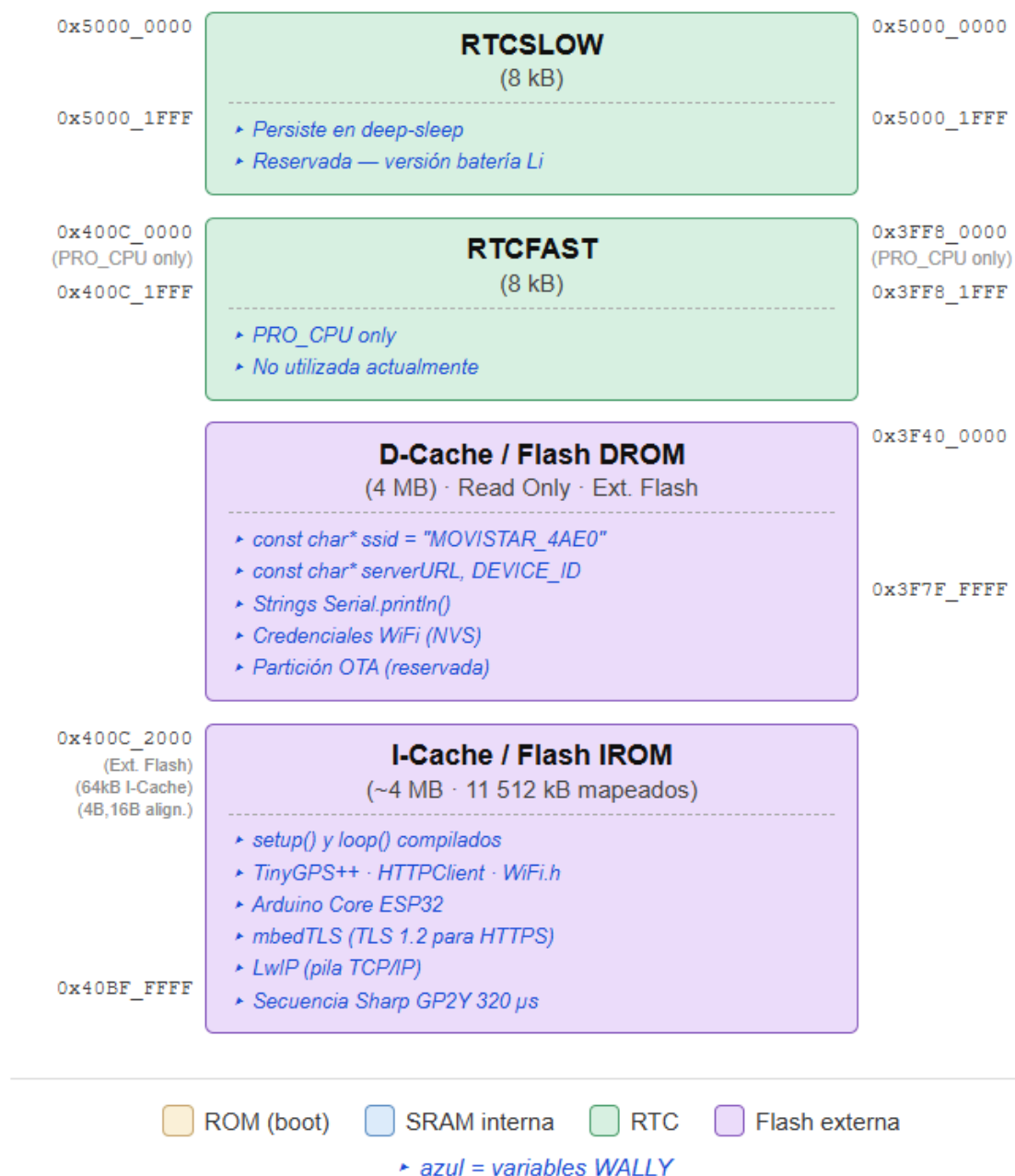
S9 - WAIT

delay(2000) — el ESP32 espera 2 segundos antes del siguiente ciclo. En versiones futuras con batería de litio, este estado se reemplazará por `esp_light_sleep_start()` para reducir el consumo de ~240 mA a menos de 1 mA durante la espera. Al terminar → regresa a S4 (el `loop()` de Arduino reinicia el ciclo de medición).

7. Mapa de Memoria (Actualizado)

A continuación el Mapa de memoria:





ROM0 — 384 kB 0x4000_0000

Memoria de fábrica, solo lectura. Contiene el bootloader que arranca el ESP32. WALLY no escribe aquí.

SRAM0 / IRAM — 128 kB 0x4007_0000

RAM interna para código de alta velocidad. Aquí corre el WiFi HAL, la pila TCP/IP y las interrupciones. Se accede sin caché — es la memoria más rápida del chip.

SRAM1 / DRAM — 128 kB 0x400A_0000

La RAM principal del programa. Aquí viven todas las variables de WALLY:

mq7Value, mq135Value, dustValue, lat, lng → en el stack

Objeto gps (TinyGPSPlus) → variable global

HTTPClient y buffers WiFi/TLS (~55 KB) → heap dinámico

SRAM2 / IPC — 200 kB 0x3FFB_0000

Buffers compartidos entre Core 0 y Core 1. Aquí se almacenan los bytes crudos del GPS antes de que TinyGPS++ los procese.

RTC SLOW — 8 kB 0x5000_0000

Sobrevive al deep-sleep. No usada ahora — reservada para cuando WALLY funcione con batería de litio.

RTC FAST — 8 kB 0x400C_0000

Similar al RTC SLOW pero solo para Core 0. No usada actualmente.

Flash DROM — 4 MB 0x3F40_0000

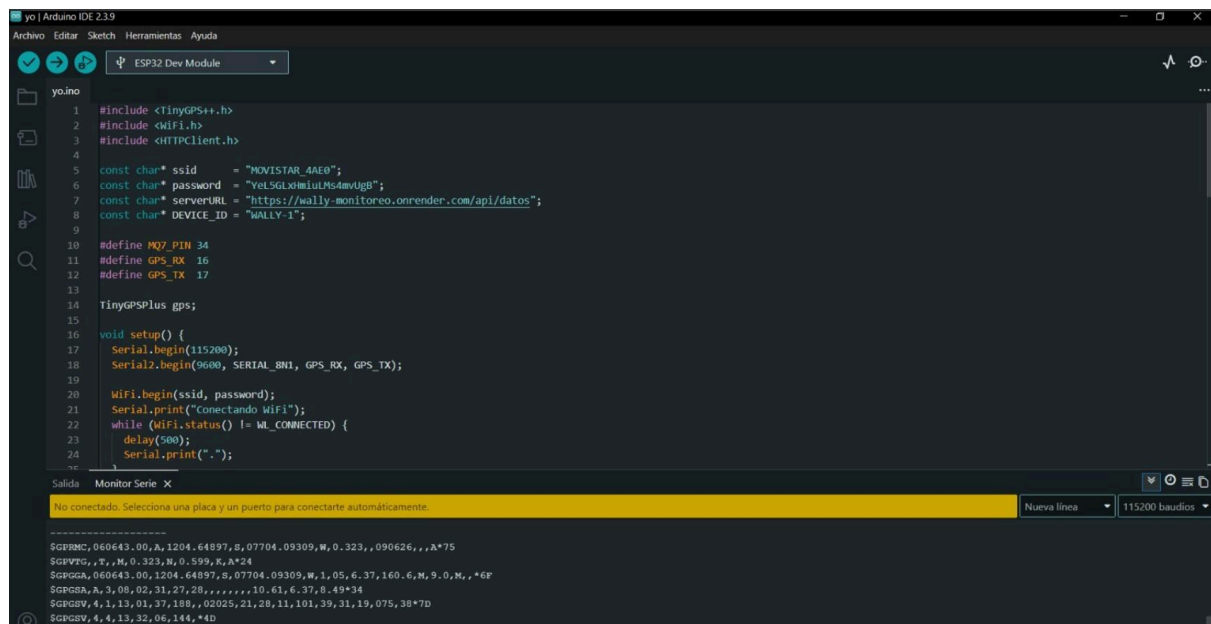
Flash externa, solo lectura por D-bus. Contiene todas las constantes: ssid, password, serverURL, DEVICE_ID y los strings de Serial.println().

Flash IROM — 4 MB 0x400C_2000

La misma Flash pero ejecutada como código por I-bus. Aquí está todo el código compilado: setup(), loop(), TinyGPS++, HTTPClient, mbedTLS (HTTPS) y LwIP (TCP/IP)

IMPLEMENTACIÓN DE LA PÁGINA WEB

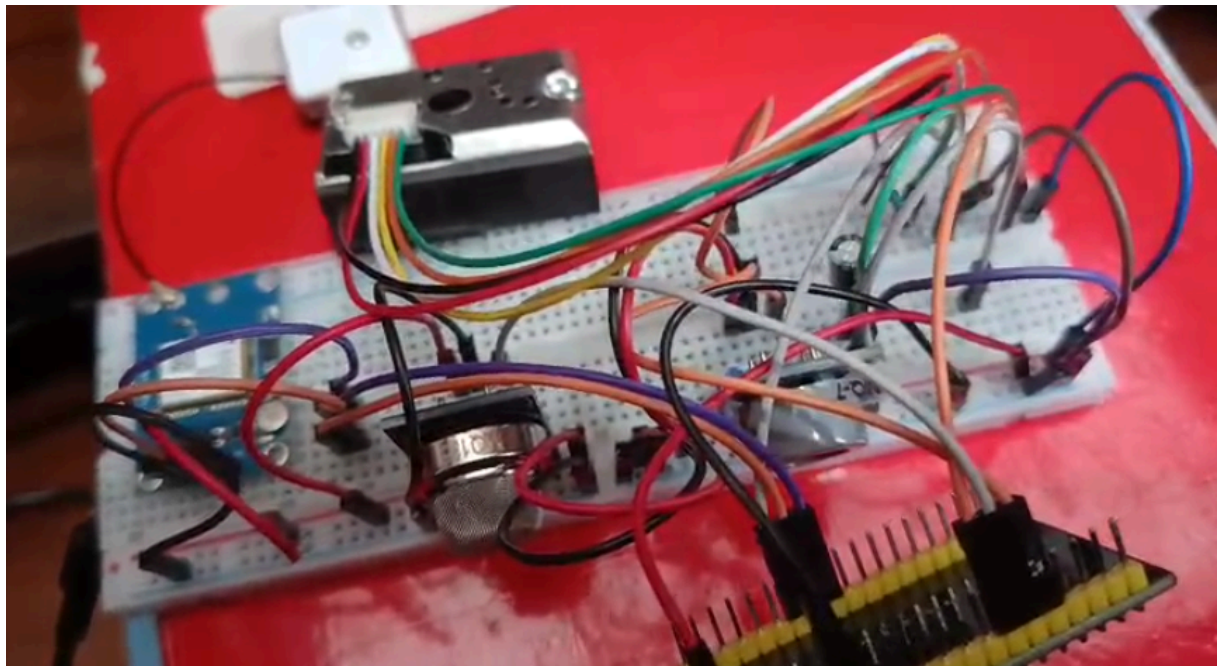
Se utilizarán librerías de ESP32 en el Arduino IDE:



```
yo | Arduino IDE 2.3.9
Archivo Editar Sketch Herramientas Ayuda
ESP32 Dev Module

yo.ino
1 #include <TinyGPS++.h>
2 #include <WiFi.h>
3 #include <HTTPClient.h>
4
5 const char* ssid = "NOVISTAR_4AE0";
6 const char* password = "YeL5GLemiuLHs4mUgB";
7 const char* serverURL = "https://wally-monitoreo.onrender.com/api/datos";
8 const char* DEVICE_ID = "WALLY-1";
9
10 #define MQ7_PIN 34
11 #define GPS_RX 16
12 #define GPS_TX 17
13
14 TinyGPSPlus gps;
15
16 void setup() {
17   Serial.begin(115200);
18   Serial2.begin(9600, SERIAL_8N1, GPS_RX, GPS_TX);
19
20   WiFi.begin(ssid, password);
21   Serial.print("Conectando WiFi");
22   while (WiFi.status() != WL_CONNECTED) {
23     delay(500);
24     Serial.print(".");
25   }
26
27   Serial.println();
28 }
29
30 void loop() {
31   gps.update();
32   if (gps.location.isUpdated()) {
33     Serial.print("Lat: ");
34     Serial.print(gps.location.lat(), 5);
35     Serial.print(" Lon: ");
36     Serial.print(gps.location.lng(), 5);
37     Serial.print(" Alt: ");
38     Serial.print(gps.altitude.meters(), 1);
39     Serial.print(" Speed: ");
40     Serial.print(gps.speed.kmh(), 1);
41     Serial.print(" Heading: ");
42     Serial.print(gps.heading(), 1);
43     Serial.print(" Temperature: ");
44     Serial.print(gps.temperature(), 1);
45     Serial.print(" Humidity: ");
46     Serial.print(gps.humidity(), 1);
47     Serial.print(" Pressure: ");
48     Serial.print(gps.pressure(), 1);
49     Serial.print(" Oxygen: ");
50     Serial.print(gps.oxygen(), 1);
51     Serial.print(" CO2: ");
52     Serial.print(gps.co2(), 1);
53     Serial.print(" NO2: ");
54     Serial.print(gps.no2(), 1);
55     Serial.print(" SO2: ");
56     Serial.print(gps.so2(), 1);
57     Serial.print(" PM2.5: ");
58     Serial.print(gps.pm25(), 1);
59     Serial.print(" PM10: ");
60     Serial.print(gps.pm10(), 1);
61     Serial.print(" AQI: ");
62     Serial.print(gps.aqi(), 1);
63     Serial.print(" UV: ");
64     Serial.print(gps.uv(), 1);
65     Serial.print(" Light: ");
66     Serial.print(gps.light(), 1);
67     Serial.print(" Sound: ");
68     Serial.print(gps.sound(), 1);
69     Serial.print(" Vibration: ");
70     Serial.print(gps.vibration(), 1);
71     Serial.print(" Acceleration: ");
72     Serial.print(gps.acceleration(), 1);
73     Serial.print(" Gyro: ");
74     Serial.print(gps.gyro(), 1);
75     Serial.print(" Magnetometer: ");
76     Serial.print(gps.magnetometer(), 1);
77     Serial.print(" Barometer: ");
78     Serial.print(gps.barometer(), 1);
79     Serial.print(" Thermistor: ");
80     Serial.print(gps.thermistor(), 1);
81     Serial.print(" I2C: ");
82     Serial.print(gps.i2c(), 1);
83     Serial.print(" SPI: ");
84     Serial.print(gps.spi(), 1);
85     Serial.print(" UART: ");
86     Serial.print(gps.uart(), 1);
87     Serial.print(" CAN: ");
88     Serial.print(gps.can(), 1);
89     Serial.print(" I2S: ");
90     Serial.print(gps.i2s(), 1);
91     Serial.print(" ADC: ");
92     Serial.print(gps.adc(), 1);
93     Serial.print(" DAC: ");
94     Serial.print(gps.dac(), 1);
95     Serial.print(" PWM: ");
96     Serial.print(gps.pwm(), 1);
97     Serial.print(" Servo: ");
98     Serial.print(gps.servo(), 1);
99     Serial.print(" Motor: ");
100    Serial.print(gps.motor(), 1);
101    Serial.print(" Relay: ");
102    Serial.print(gps.relay(), 1);
103    Serial.print(" Switch: ");
104    Serial.print(gps.switch(), 1);
105    Serial.print(" Button: ");
106    Serial.print(gps.button(), 1);
107    Serial.print(" Joystick: ");
108    Serial.print(gps.joystick(), 1);
109    Serial.print(" Potentiometer: ");
110    Serial.print(gps.potentiometer(), 1);
111    Serial.print(" Encoder: ");
112    Serial.print(gps.encoder(), 1);
113    Serial.print(" Touchscreen: ");
114    Serial.print(gps.touchscreen(), 1);
115    Serial.print(" Proximity: ");
116    Serial.print(gps.proximity(), 1);
117    Serial.print(" Infrared: ");
118    Serial.print(gps.infrared(), 1);
119    Serial.print(" Ultrasonic: ");
120    Serial.print(gps.ultrasonic(), 1);
121    Serial.print(" Sonar: ");
122    Serial.print(gps.sonar(), 1);
123    Serial.print(" Laser: ");
124    Serial.print(gps.laser(), 1);
125    Serial.print(" Radar: ");
126    Serial.print(gps.radar(), 1);
127    Serial.print(" Lidar: ");
128    Serial.print(gps.lidar(), 1);
129    Serial.print(" Camera: ");
130    Serial.print(gps.camera(), 1);
131    Serial.print(" Microphone: ");
132    Serial.print(gps.microphone(), 1);
133    Serial.print(" Speaker: ");
134    Serial.print(gps.speaker(), 1);
135    Serial.print(" Buzzer: ");
136    Serial.print(gps.buzzer(), 1);
137    Serial.print(" LED: ");
138    Serial.print(gps.led(), 1);
139    Serial.print(" RGB: ");
140    Serial.print(gps.rgb(), 1);
141    Serial.print(" NeoPixel: ");
142    Serial.print(gps.neopixel(), 1);
143    Serial.print(" WS2812: ");
144    Serial.print(gps.ws2812(), 1);
145    Serial.print(" APA102: ");
146    Serial.print(gps.apa102(), 1);
147    Serial.print(" SK6812: ");
148    Serial.print(gps.sk6812(), 1);
149    Serial.print(" TM1637: ");
150    Serial.print(gps.tm1637(), 1);
151    Serial.print(" SSD1306: ");
152    Serial.print(gps.ssd1306(), 1);
153    Serial.print(" ST7735: ");
154    Serial.print(gps.st7735(), 1);
155    Serial.print(" ILI9341: ");
156    Serial.print(gps.ili9341(), 1);
157    Serial.print(" TFT: ");
158    Serial.print(gps.tft(), 1);
159    Serial.print(" LCD: ");
160    Serial.print(gps.lcd(), 1);
161    Serial.print(" Keypad: ");
162    Serial.print(gps.keypad(), 1);
163    Serial.print(" DHT: ");
164    Serial.print(gps.dht(), 1);
165    Serial.print(" DHT22: ");
166    Serial.print(gps.dht22(), 1);
167    Serial.print(" DHT11: ");
168    Serial.print(gps.dht11(), 1);
169    Serial.print(" DS18B20: ");
170    Serial.print(gps.ds18b20(), 1);
171    Serial.print(" LM75: ");
172    Serial.print(gps.lm75(), 1);
173    Serial.print(" MCP9802: ");
174    Serial.print(gps.mcp9802(), 1);
175    Serial.print(" TMP36: ");
176    Serial.print(gps.tmp36(), 1);
177    Serial.print(" 1-Wire: ");
178    Serial.print(gps.one_wire(), 1);
179    Serial.print(" I2C: ");
180    Serial.print(gps.i2c(), 1);
181    Serial.print(" SPI: ");
182    Serial.print(gps.spi(), 1);
183    Serial.print(" UART: ");
184    Serial.print(gps.uart(), 1);
185    Serial.print(" CAN: ");
186    Serial.print(gps.can(), 1);
187    Serial.print(" I2S: ");
188    Serial.print(gps.i2s(), 1);
189    Serial.print(" ADC: ");
190    Serial.print(gps.adc(), 1);
191    Serial.print(" DAC: ");
192    Serial.print(gps.dac(), 1);
193    Serial.print(" PWM: ");
194    Serial.print(gps.pwm(), 1);
195    Serial.print(" Servo: ");
196    Serial.print(gps.servo(), 1);
197    Serial.print(" Motor: ");
198    Serial.print(gps.motor(), 1);
199    Serial.print(" Relay: ");
200    Serial.print(gps.relay(), 1);
201    Serial.print(" Switch: ");
202    Serial.print(gps.switch(), 1);
203    Serial.print(" Button: ");
204    Serial.print(gps.button(), 1);
205    Serial.print(" Joystick: ");
206    Serial.print(gps.joystick(), 1);
207    Serial.print(" Potentiometer: ");
208    Serial.print(gps.potentiometer(), 1);
209    Serial.print(" Encoder: ");
210    Serial.print(gps.encoder(), 1);
211    Serial.print(" Touchscreen: ");
212    Serial.print(gps.touchscreen(), 1);
213    Serial.print(" Proximity: ");
214    Serial.print(gps.proximity(), 1);
215    Serial.print(" Infrared: ");
216    Serial.print(gps.infrared(), 1);
217    Serial.print(" Ultrasonic: ");
218    Serial.print(gps.ultrasonic(), 1);
219    Serial.print(" Sonar: ");
220    Serial.print(gps.sonar(), 1);
221    Serial.print(" Laser: ");
222    Serial.print(gps.laser(), 1);
223    Serial.print(" Radar: ");
224    Serial.print(gps.radar(), 1);
225    Serial.print(" Lidar: ");
226    Serial.print(gps.lidar(), 1);
227    Serial.print(" Camera: ");
228    Serial.print(gps.camera(), 1);
229    Serial.print(" Microphone: ");
230    Serial.print(gps.microphone(), 1);
231    Serial.print(" Speaker: ");
232    Serial.print(gps.speaker(), 1);
233    Serial.print(" Buzzer: ");
234    Serial.print(gps.buzzer(), 1);
235    Serial.print(" LED: ");
236    Serial.print(gps.led(), 1);
237    Serial.print(" RGB: ");
238    Serial.print(gps.rgb(), 1);
239    Serial.print(" NeoPixel: ");
240    Serial.print(gps.neopixel(), 1);
241    Serial.print(" WS2812: ");
242    Serial.print(gps.ws2812(), 1);
243    Serial.print(" APA102: ");
244    Serial.print(gps.apa102(), 1);
245    Serial.print(" SK6812: ");
246    Serial.print(gps.sk6812(), 1);
247    Serial.print(" TM1637: ");
248    Serial.print(gps.tm1637(), 1);
249    Serial.print(" SSD1306: ");
250    Serial.print(gps.ssd1306(), 1);
251    Serial.print(" ST7735: ");
252    Serial.print(gps.st7735(), 1);
253    Serial.print(" ILI9341: ");
254    Serial.print(gps.ili9341(), 1);
255    Serial.print(" TFT: ");
256    Serial.print(gps.tft(), 1);
257    Serial.print(" LCD: ");
258    Serial.print(gps.lcd(), 1);
259    Serial.print(" Keypad: ");
260    Serial.print(gps.keypad(), 1);
261    Serial.print(" DHT: ");
262    Serial.print(gps.dht(), 1);
263    Serial.print(" DHT22: ");
264    Serial.print(gps.dht22(), 1);
265    Serial.print(" DHT11: ");
266    Serial.print(gps.dht11(), 1);
267    Serial.print(" DS18B20: ");
268    Serial.print(gps.ds18b20(), 1);
269    Serial.print(" LM75: ");
270    Serial.print(gps.lm75(), 1);
271    Serial.print(" MCP9802: ");
272    Serial.print(gps.mcp9802(), 1);
273    Serial.print(" TMP36: ");
274    Serial.print(gps.tmp36(), 1);
275    Serial.print(" 1-Wire: ");
276    Serial.print(gps.one_wire(), 1);
277    Serial.print(" I2C: ");
278    Serial.print(gps.i2c(), 1);
279    Serial.print(" SPI: ");
280    Serial.print(gps.spi(), 1);
281    Serial.print(" UART: ");
282    Serial.print(gps.uart(), 1);
283    Serial.print(" CAN: ");
284    Serial.print(gps.can(), 1);
285    Serial.print(" I2S: ");
286    Serial.print(gps.i2s(), 1);
287    Serial.print(" ADC: ");
288    Serial.print(gps.adc(), 1);
289    Serial.print(" DAC: ");
290    Serial.print(gps.dac(), 1);
291    Serial.print(" PWM: ");
292    Serial.print(gps.pwm(), 1);
293    Serial.print(" Servo: ");
294    Serial.print(gps.servo(), 1);
295    Serial.print(" Motor: ");
296    Serial.print(gps.motor(), 1);
297    Serial.print(" Relay: ");
298    Serial.print(gps.relay(), 1);
299    Serial.print(" Switch: ");
300    Serial.print(gps.switch(), 1);
301    Serial.print(" Button: ");
302    Serial.print(gps.button(), 1);
303    Serial.print(" Joystick: ");
304    Serial.print(gps.joystick(), 1);
305    Serial.print(" Potentiometer: ");
306    Serial.print(gps.potentiometer(), 1);
307    Serial.print(" Encoder: ");
308    Serial.print(gps.encoder(), 1);
309    Serial.print(" Touchscreen: ");
310    Serial.print(gps.touchscreen(), 1);
311    Serial.print(" Proximity: ");
312    Serial.print(gps.proximity(), 1);
313    Serial.print(" Infrared: ");
314    Serial.print(gps.infrared(), 1);
315    Serial.print(" Ultrasonic: ");
316    Serial.print(gps.ultrasonic(), 1);
317    Serial.print(" Sonar: ");
318    Serial.print(gps.sonar(), 1);
319    Serial.print(" Laser: ");
320    Serial.print(gps.laser(), 1);
321    Serial.print(" Radar: ");
322    Serial.print(gps.radar(), 1);
323    Serial.print(" Lidar: ");
324    Serial.print(gps.lidar(), 1);
325    Serial.print(" Camera: ");
326    Serial.print(gps.camera(), 1);
327    Serial.print(" Microphone: ");
328    Serial.print(gps.microphone(), 1);
329    Serial.print(" Speaker: ");
330    Serial.print(gps.speaker(), 1);
331    Serial.print(" Buzzer: ");
332    Serial.print(gps.buzzer(), 1);
333    Serial.print(" LED: ");
334    Serial.print(gps.led(), 1);
335    Serial.print(" RGB: ");
336    Serial.print(gps.rgb(), 1);
337    Serial.print(" NeoPixel: ");
338    Serial.print(gps.neopixel(), 1);
339    Serial.print(" WS2812: ");
340    Serial.print(gps.ws2812(), 1);
341    Serial.print(" APA102: ");
342    Serial.print(gps.apa102(), 1);
343    Serial.print(" SK6812: ");
344    Serial.print(gps.sk6812(), 1);
345    Serial.print(" TM1637: ");
346    Serial.print(gps.tm1637(), 1);
347    Serial.print(" SSD1306: ");
348    Serial.print(gps.ssd1306(), 1);
349    Serial.print(" ST7735: ");
350    Serial.print(gps.st7735(), 1);
351    Serial.print(" ILI9341: ");
352    Serial.print(gps.ili9341(), 1);
353    Serial.print(" TFT: ");
354    Serial.print(gps.tft(), 1);
355    Serial.print(" LCD: ");
356    Serial.print(gps.lcd(), 1);
357    Serial.print(" Keypad: ");
358    Serial.print(gps.keypad(), 1);
359    Serial.print(" DHT: ");
360    Serial.print(gps.dht(), 1);
361    Serial.print(" DHT22: ");
362    Serial.print(gps.dht22(), 1);
363    Serial.print(" DHT11: ");
364    Serial.print(gps.dht11(), 1);
365    Serial.print(" DS18B20: ");
366    Serial.print(gps.ds18b20(), 1);
367    Serial.print(" LM75: ");
368    Serial.print(gps.lm75(), 1);
369    Serial.print(" MCP9802: ");
370    Serial.print(gps.mcp9802(), 1);
371    Serial.print(" TMP36: ");
372    Serial.print(gps.tmp36(), 1);
373    Serial.print(" 1-Wire: ");
374    Serial.print(gps.one_wire(), 1);
375    Serial.print(" I2C: ");
376    Serial.print(gps.i2c(), 1);
377    Serial.print(" SPI: ");
378    Serial.print(gps.spi(), 1);
379    Serial.print(" UART: ");
380    Serial.print(gps.uart(), 1);
381    Serial.print(" CAN: ");
382    Serial.print(gps.can(), 1);
383    Serial.print(" I2S: ");
384    Serial.print(gps.i2s(), 1);
385    Serial.print(" ADC: ");
386    Serial.print(gps.adc(), 1);
387    Serial.print(" DAC: ");
388    Serial.print(gps.dac(), 1);
389    Serial.print(" PWM: ");
390    Serial.print(gps.pwm(), 1);
391    Serial.print(" Servo: ");
392    Serial.print(gps.servo(), 1);
393    Serial.print(" Motor: ");
394    Serial.print(gps.motor(), 1);
395    Serial.print(" Relay: ");
396    Serial.print(gps.relay(), 1);
397    Serial.print(" Switch: ");
398    Serial.print(gps.switch(), 1);
399    Serial.print(" Button: ");
400    Serial.print(gps.button(), 1);
401    Serial.print(" Joystick: ");
402    Serial.print(gps.joystick(), 1);
403    Serial.print(" Potentiometer: ");
404    Serial.print(gps.potentiometer(), 1);
405    Serial.print(" Encoder: ");
406    Serial.print(gps.encoder(), 1);
407    Serial.print(" Touchscreen: ");
408    Serial.print(gps.touchscreen(), 1);
409    Serial.print(" Proximity: ");
410    Serial.print(gps.proximity(), 1);
411    Serial.print(" Infrared: ");
412    Serial.print(gps.infrared(), 1);
413    Serial.print(" Ultrasonic: ");
414    Serial.print(gps.ultrasonic(), 1);
415    Serial.print(" Sonar: ");
416    Serial.print(gps.sonar(), 1);
417    Serial.print(" Laser: ");
418    Serial.print(gps.laser(), 1);
419    Serial.print(" Radar: ");
420    Serial.print(gps.radar(), 1);
421    Serial.print(" Lidar: ");
422    Serial.print(gps.lidar(), 1);
423    Serial.print(" Camera: ");
424    Serial.print(gps.camera(), 1);
425    Serial.print(" Microphone: ");
426    Serial.print(gps.microphone(), 1);
427    Serial.print(" Speaker: ");
428    Serial.print(gps.speaker(), 1);
429    Serial.print(" Buzzer: ");
430    Serial.print(gps.buzzer(), 1);
431    Serial.print(" LED: ");
432    Serial.print(gps.led(), 1);
433    Serial.print(" RGB: ");
434    Serial.print(gps.rgb(), 1);
435    Serial.print(" NeoPixel: ");
436    Serial.print(gps.neopixel(), 1);
437    Serial.print(" WS2812: ");
438    Serial.print(gps.ws2812(), 1);
439    Serial.print(" APA102: ");
440    Serial.print(gps.apa102(), 1);
441    Serial.print(" SK6812: ");
442    Serial.print(gps.sk6812(), 1);
443    Serial.print(" TM1637: ");
444    Serial.print(gps.tm1637(), 1);
445    Serial.print(" SSD1306: ");
446    Serial.print(gps.ssd1306(), 1);
447    Serial.print(" ST7735: ");
448    Serial.print(gps.st7735(), 1);
449    Serial.print(" ILI9341: ");
450    Serial.print(gps.ili9341(), 1);
451    Serial.print(" TFT: ");
452    Serial.print(gps.tft(), 1);
453    Serial.print(" LCD: ");
454    Serial.print(gps.lcd(), 1);
455    Serial.print(" Keypad: ");
456    Serial.print(gps.keypad(), 1);
457    Serial.print(" DHT: ");
458    Serial.print(gps.dht(), 1);
459    Serial.print(" DHT22: ");
460    Serial.print(gps.dht22(), 1);
461    Serial.print(" DHT11: ");
462    Serial.print(gps.dht11(), 1);
463    Serial.print(" DS18B20: ");
464    Serial.print(gps.ds18b20(), 1);
465    Serial.print(" LM75: ");
466    Serial.print(gps.lm75(), 1);
467    Serial.print(" MCP9802: ");
468    Serial.print(gps.mcp9802(), 1);
469    Serial.print(" TMP36: ");
470    Serial.print(gps.tmp36(), 1);
471    Serial.print(" 1-Wire: ");
472    Serial.print(gps.one_wire(), 1);
473    Serial.print(" I2C: ");
474    Serial.print(gps.i2c(), 1);
475    Serial.print(" SPI: ");
476    Serial.print(gps.spi(), 1);
477    Serial.print(" UART: ");
478    Serial.print(gps.uart(), 1);
479    Serial.print(" CAN: ");
480    Serial.print(gps.can(), 1);
481    Serial.print(" I2S: ");
482    Serial.print(gps.i2s(), 1);
483    Serial.print(" ADC: ");
484    Serial.print(gps.adc(), 1);
485    Serial.print(" DAC: ");
486    Serial.print(gps.dac(), 1);
487    Serial.print(" PWM: ");
488    Serial.print(gps.pwm(), 1);
489    Serial.print(" Servo: ");
490    Serial.print(gps.servo(), 1);
491    Serial.print(" Motor: ");
492    Serial.print(gps.motor(), 1);
493    Serial.print(" Relay: ");
494    Serial.print(gps.relay(), 1);
495    Serial.print(" Switch: ");
496    Serial.print(gps.switch(), 1);
497    Serial.print(" Button: ");
498    Serial.print(gps.button(), 1);
499    Serial.print(" Joystick: ");
500    Serial.print(gps.joystick(), 1);
501    Serial.print(" Potentiometer: ");
502    Serial.print(gps.potentiometer(), 1);
503    Serial.print(" Encoder: ");
504    Serial.print(gps.encoder(), 1);
505    Serial.print(" Touchscreen: ");
506    Serial.print(gps.touchscreen(), 1);
507    Serial.print(" Proximity: ");
508    Serial.print(gps.proximity(), 1);
509    Serial.print(" Infrared: ");
510    Serial.print(gps.infrared(), 1);
511    Serial.print(" Ultrasonic: ");
512    Serial.print(gps.ultrasonic(), 1);
513    Serial.print(" Sonar: ");
514    Serial.print(gps.sonar(), 1);
515    Serial.print(" Laser: ");
516    Serial.print(gps.laser(), 1);
517    Serial.print(" Radar: ");
518    Serial.print(gps.radar(), 1);
519    Serial.print(" Lidar: ");
520    Serial.print(gps.lidar(), 1);
521    Serial.print(" Camera: ");
522    Serial.print(gps.camera(), 1);
523    Serial.print(" Microphone: ");
524    Serial.print(gps.microphone(), 1);
525    Serial.print(" Speaker: ");
526    Serial.print(gps.speaker(), 1);
527    Serial.print(" Buzzer: ");
528    Serial.print(gps.buzzer(), 1);
529    Serial.print(" LED: ");
530    Serial.print(gps.led(), 1);
531    Serial.print(" RGB: ");
532    Serial.print(gps.rgb(), 1);
533    Serial.print(" NeoPixel: ");
534    Serial.print(gps.neopixel(), 1);
535    Serial.print(" WS2812: ");
536    Serial.print(gps.ws2812(), 1);
537    Serial.print(" APA102: ");
538    Serial.print(gps.apa102(), 1);
539    Serial.print(" SK6812: ");
540    Serial.print(gps.sk6812(), 1);
541    Serial.print(" TM1637: ");
542    Serial.print(gps.tm1637(), 1);
543    Serial.print(" SSD1306: ");
544    Serial.print(gps.ssd1306(), 1);
545    Serial.print(" ST7735: ");
546    Serial.print(gps.st7735(), 1);
547    Serial.print(" ILI9341: ");
548    Serial.print(gps.ili9341(), 1);
549    Serial.print(" TFT: ");
550    Serial.print(gps.tft(), 1);
551    Serial.print(" LCD: ");
552    Serial.print(gps.lcd(), 1);
553    Serial.print(" Keypad: ");
554    Serial.print(gps.keypad(), 1);
555    Serial.print(" DHT: ");
556    Serial.print(gps.dht(), 1);
557    Serial.print(" DHT22: ");
558    Serial.print(gps.dht22(), 1);
559    Serial.print(" DHT11: ");
560    Serial.print(gps.dht11(), 1);
561    Serial.print(" DS18B20: ");
562    Serial.print(gps.ds18b20(), 1);
563    Serial.print(" LM75: ");
564    Serial.print(gps.lm75(), 1);
565    Serial.print(" MCP9802: ");
566    Serial.print(gps.mcp9802(), 1);
567    Serial.print(" TMP36: ");
568    Serial.print(gps.tmp36(), 1);
569    Serial.print(" 1-Wire: ");
570    Serial.print(gps.one_wire(), 1);
571    Serial.print(" I2C: ");
572    Serial.print(gps.i2c(), 1);
573    Serial.print(" SPI: ");
574    Serial.print(gps.spi(), 1);
575    Serial.print(" UART: ");
576    Serial.print(gps.uart(), 1);
577    Serial.print(" CAN: ");
578    Serial.print(gps.can(), 1);
579    Serial.print(" I2S: ");
580    Serial.print(gps.i2s(), 1);
581    Serial.print(" ADC: ");
582    Serial.print(gps.adc(), 1);
583    Serial.print(" DAC: ");
584    Serial.print(gps.dac(), 1);
585    Serial.print(" PWM: ");
586    Serial.print(gps.pwm(), 1);
587    Serial.print(" Servo: ");
588    Serial.print(gps.servo(), 1);
589    Serial.print(" Motor: ");
590    Serial.print(gps.motor(), 1);
591    Serial.print(" Relay: ");
592    Serial.print(gps.relay(), 1);
593    Serial.print(" Switch: ");
594    Serial.print(gps.switch(), 1);
595    Serial.print(" Button: ");
596    Serial.print(gps.button(), 1);
597    Serial.print(" Joystick: ");
598    Serial.print(gps.joystick(), 1);
599    Serial.print(" Potentiometer: ");
600    Serial.print(gps.potentiometer(), 1);
601    Serial.print(" Encoder: ");
602    Serial.print(gps.encoder(), 1);
603    Serial.print(" Touchscreen: ");
604    Serial.print(gps.touchscreen(), 1);
605    Serial.print(" Proximity: ");
606    Serial.print(gps.proximity(), 1);
607    Serial.print(" Infrared: ");
608    Serial.print(gps.infrared(), 1);
609    Serial.print(" Ultrasonic: ");
610    Serial.print(gps.ultrasonic(), 1);
611    Serial.print(" Sonar: ");
612    Serial.print(gps.sonar(), 1);
613    Serial.print(" Laser: ");
614    Serial.print(gps.laser(), 1);
615    Serial.print(" Radar: ");
616    Serial.print(gps.radar(), 1);
617    Serial.print(" Lidar: ");
618    Serial.print(gps.lidar(), 1);
619    Serial.print(" Camera: ");
620    Serial.print(gps.camera(), 1);
621    Serial.print(" Microphone: ");
622    Serial.print(gps.microphone(), 1);
623    Serial.print(" Speaker: ");
624    Serial.print(gps.speaker(), 1);
625    Serial.print(" Buzzer: ");
626    Serial.print(gps.buzzer(), 1);
627    Serial.print(" LED: ");
628    Serial.print(gps.led(), 1);
629    Serial.print(" RGB: ");
630    Serial.print(gps.rgb(), 1);
631    Serial.print(" NeoPixel: ");
632    Serial.print(gps.neopixel(), 1);
633    Serial.print(" WS2812: ");
634    Serial.print(gps.ws2812(), 1);
635    Serial.print(" APA102: ");
636    Serial.print(gps.apa102(), 1);
637    Serial.print(" SK6812: ");
638    Serial.print(gps.sk6812(), 1);
639    Serial.print(" TM1637: ");
640    Serial.print(gps.tm1637(), 1);
641    Serial.print(" SSD1306: ");
642    Serial.print(gps.ssd1306(), 1);
643    Serial.print(" ST7735: ");
644    Serial.print(gps.st7735(), 1);
645    Serial.print(" ILI9341: ");
646    Serial.print(gps.ili9341(), 1);
647    Serial.print(" TFT: ");
648    Serial.print(gps.tft(), 1);
649    Serial.print(" LCD: ");
650    Serial.print(gps.lcd(), 1);
651    Serial.print(" Keypad: ");
652    Serial.print(gps.keypad(), 1);
653    Serial.print(" DHT: ");
654    Serial.print(gps.dht(), 1);
655    Serial.print(" DHT22: ");
656    Serial.print(gps.dht22(), 1);
657    Serial.print(" DHT11: ");
658    Serial.print(gps.dht11(), 1);
659    Serial.print(" DS18B20: ");
660    Serial.print(gps.ds18b20(), 1);
661    Serial.print(" LM75: ");
662    Serial.print(gps.lm75(), 1);
663    Serial.print(" MCP9802: ");
664    Serial.print(gps.mcp9802(), 1);
665    Serial.print(" TMP36: ");
666    Serial.print(gps.tmp36(), 1);
667    Serial.print(" 1-Wire: ");
668    Serial.print(gps.one_wire(), 1);
669    Serial.print(" I2C: ");
670    Serial.print(gps.i2c(), 1);
671    Serial.print(" SPI: ");
672    Serial.print(gps.spi(), 1);
673    Serial.print(" UART: ");
674    Serial.print(gps.uart(), 1);
675    Serial.print(" CAN: ");
676    Serial.print(gps.can(), 1);
677    Serial.print(" I2S: ");
678    Serial.print(gps.i2s(), 1);
679    Serial.print(" ADC: ");
680    Serial.print(gps.adc(), 1);
681    Serial.print(" DAC: ");
682    Serial.print(gps.dac(), 1);
683    Serial.print(" PWM: ");
684    Serial.print(gps.pwm(), 1);
685    Serial.print(" Servo: ");
686    Serial.print(gps.servo(), 1);
687    Serial.print(" Motor: ");
688    Serial.print(gps.motor(), 1);
689    Serial.print(" Relay: ");
690    Serial.print(gps.relay(), 1);
691    Serial.print(" Switch: ");
692    Serial.print(gps.switch(), 1);
693    Serial.print(" Button: ");
694    Serial.print(gps.button(), 1);
695    Serial.print(" Joystick: ");
696    Serial.print(gps.joystick(), 1);
697    Serial.print(" Potentiometer: ");
698    Serial.print(gps.potentiometer(), 1);
699    Serial.print(" Encoder: ");
700    Serial.print(gps.encoder(), 1);
701    Serial.print(" Touchscreen: ");
702    Serial.print(gps.touchscreen(), 1);
703    Serial.print(" Proximity: ");
704    Serial.print(gps.proximity(), 1);
705    Serial.print(" Infrared: ");
706    Serial.print(gps.infrared(), 1);
707    Serial.print(" Ultrasonic: ");
708    Serial.print(gps.ultrasonic(), 1);
709    Serial.print(" Sonar: ");
710    Serial.print(gps.sonar(), 1);
711    Serial.print(" Laser: ");
712    Serial.print(gps.laser(), 1);
713    Serial.print(" Radar: ");
714    Serial.print(gps.radar(), 1);
715    Serial.print(" Lidar: ");
716    Serial.print(gps.lidar(), 1);
717    Serial.print(" Camera: ");
718    Serial.print(gps.camera(), 1);
719    Serial.print(" Microphone: ");
720    Serial.print(gps.microphone(), 1);
721    Serial.print(" Speaker: ");
722    Serial.print(gps.speaker(), 1);
723    Serial.print(" Buzzer: ");
724    Serial.print(gps.buzzer(), 1);
725    Serial.print(" LED: ");
726    Serial.print(gps.led(), 1);
727    Serial.print(" RGB: ");
728    Serial.print(gps.rgb(), 1);
729    Serial.print(" NeoPixel: ");
730    Serial.print(gps.neopixel(), 1);
731    Serial.print(" WS2812: ");
732    Serial.print(gps.ws2812(), 1);
733    Serial.print(" APA102: ");
734    Serial.print(gps.apa102(), 1);
735    Serial.print(" SK6812: ");
736    Serial.print(gps.sk6812(), 1);
737    Serial.print(" TM1637: ");
738    Serial.print(gps.tm1637(), 1);
739    Serial.print(" SSD1306: ");
740    Serial.print(gps.ssd1306(), 1);
741    Serial.print(" ST7735: ");
742    Serial.print(gps.st7735(), 1);
743    Serial.print(" ILI9341: ");
744    Serial.print(gps.ili9341(), 1);
745    Serial.print(" TFT: ");
746    Serial.print(gps.tft(), 1);
747    Serial.print(" LCD: ");
748    Serial.print(gps.lcd(), 1);
749    Serial.print(" Keypad: ");
750    Serial.print(gps.keypad(), 1);
751    Serial.print(" DHT: ");
752    Serial.print(gps.dht(), 1);
753    Serial.print(" DHT22: ");
754    Serial.print(gps.dht22(), 1);
755    Serial.print(" DHT11: ");
756    Serial.print(gps.dht11(), 1);
757    Serial.print(" DS18B20: ");
758    Serial.print(gps.ds18b20(), 1);
759    Serial.print(" LM75: ");
760    Serial.print(gps.lm75(), 1);
761    Serial.print(" MCP9802: ");
762    Serial.print(gps.mcp9802(), 1);
763    Serial.print(" TMP36: ");
764    Serial.print(gps.tmp36(), 1);
765    Serial.print(" 1-Wire: ");
766    Serial.print(gps.one_wire(), 1);
767    Serial.print(" I2C: ");
768    Serial.print(gps.i2c(), 1);
769    Serial.print(" SPI: ");
770    Serial.print(gps.spi(), 1);
771    Serial.print(" UART: ");
772    Serial.print(gps.uart(), 1);
773    Serial.print(" CAN: ");
774    Serial.print(gps.can(), 1);
775    Serial.print(" I2S: ");
776    Serial.print(gps.i2s(), 1);
777    Serial.print(" ADC: ");
778    Serial.print(gps.adc(), 1);
779    Serial.print(" DAC: ");
780    Serial.print(gps.dac(), 1);
781    Serial.print(" PWM: ");
782    Serial.print(gps.pwm(), 1);
783    Serial.print(" Servo: ");
784    Serial.print(gps.servo(), 1);
785    Serial.print(" Motor: ");
786    Serial.print(gps.motor(), 1);
787    Serial.print(" Relay: ");
788    Serial.print(gps.relay(), 1);
789    Serial.print(" Switch: ");
790    Serial.print(gps.switch(), 1);
791    Serial.print(" Button: ");
792    Serial.print(gps.button(), 1);
793    Serial.print(" Joystick: ");
794    Serial.print(gps.joystick(), 1);
795    Serial.print(" Potentiometer: ");
796    Serial.print(gps.potentiometer(), 1);
797    Serial.print(" Encoder: ");
798    Serial.print(gps.encoder(), 1);
799    Serial.print(" Touchscreen: ");
800    Serial.print(gps.touchscreen(), 1);
801    Serial.print(" Proximity: ");
802    Serial.print(gps.proximity(), 1);
803    Serial.print(" Infrared: ");
804    Serial.print(gps.infrared(), 1);
805    Serial.print(" Ultrasonic: ");
806    Serial.print(gps.ultrasonic(), 1);
807    Serial.print(" Sonar: ");
808    Serial.print(gps.sonar(), 1);
809    Serial.print(" Laser: ");
810    Serial.print(gps.laser(), 1);
811    Serial.print(" Radar: ");
812    Serial.print(gps.radar(), 1);
813    Serial.print(" Lidar: ");
814    Serial.print(gps.lidar(), 1);
815    Serial.print(" Camera: ");
816    Serial.print(gps.camera(), 1);
817    Serial.print(" Microphone: ");
818    Serial.print(gps.microphone(), 1);
819    Serial.print(" Speaker: ");
820    Serial.print(gps.speaker(), 1);
821    Serial.print(" Buzzer: ");
822    Serial.print(gps.buzzer(), 1);
823    Serial.print(" LED: ");
824    Serial.print(gps.led(), 1);
825    Serial.print(" RGB: ");
826    Serial.print(gps.rgb(), 1);
827    Serial.print(" NeoPixel: ");
828    Serial.print(gps.neopixel(), 1);
829    Serial.print(" WS2812: ");
830    Serial.print(gps.ws2812(), 1);
831    Serial.print(" APA102: ");
832    Serial.print(gps.apa102(), 1);
833    Serial.print(" SK6812: ");
834    Serial.print(gps.sk6812(), 1);
835    Serial.print(" TM1637: ");
836    Serial.print(gps.tm1637(), 1);
837    Serial.print(" SSD1306: ");
838    Serial.print(gps.ssd1306(), 1);
839    Serial.print(" ST7735: ");
840    Serial.print(gps.st7735(), 1);
841    Serial.print(" ILI9341: ");
842    Serial.print(gps.ili9341(), 1);
843    Serial.print(" TFT: ");
844    Serial.print(gps.tft(), 1);
845    Serial.print(" LCD: ");
846    Serial.print(gps.lcd(), 
```

Anexo



```
----- SENSORZS -----  
Sharp (polvo raw): 27  
MQ-7 (CO): 609  
MQ-135 (aire): 431  
GPS: buscando señal...  
-----  
  
----- SENSORES -----  
Sharp (polvo raw): 25  
MQ-7 (CO): 619  
MQ-135 (aire): 429  
GPS: buscando señal...  
-----  
  
----- SENSORES -----  
Sharp (polvo raw): 30  
MQ-7 (CO): 613  
MQ-135 (aire): 426  
GPS: buscando señal...  
-----
```